

Backtracking

Today's Content

- Revision
- What is Backtracking?
- Valid Paranthesis
- All subsets
- All permutations.

Bhargav

- Google (2+ yrs)
- Scaler (1+ yrs)
- 5 yrs experience

Quiz 1:

```
int magicfunc(int N) {  
    if (N == 0) return 0;  
    return magicfunc(N/2) * 10 + (N%2);  
}
```

$$\underline{N=7} \rightarrow \text{magicfunc}(7) \rightarrow 11 * 10 + 7 \% 2 = \boxed{111}$$

↓ ↑ 11

$$\text{magicfunc}(3) \rightarrow 1 * 10 + 3 \% 2;$$

↓ ↑ 1

$$\text{magicfunc}(1) \rightarrow 0 * 10 + 1 \% 2;$$

↓ ↑ 0

$$\text{magicfunc}(0)$$

$$N \rightarrow N/2 \rightarrow N/4 \rightarrow N/8 \dots 1 \rightarrow 0 \quad \} \log N$$

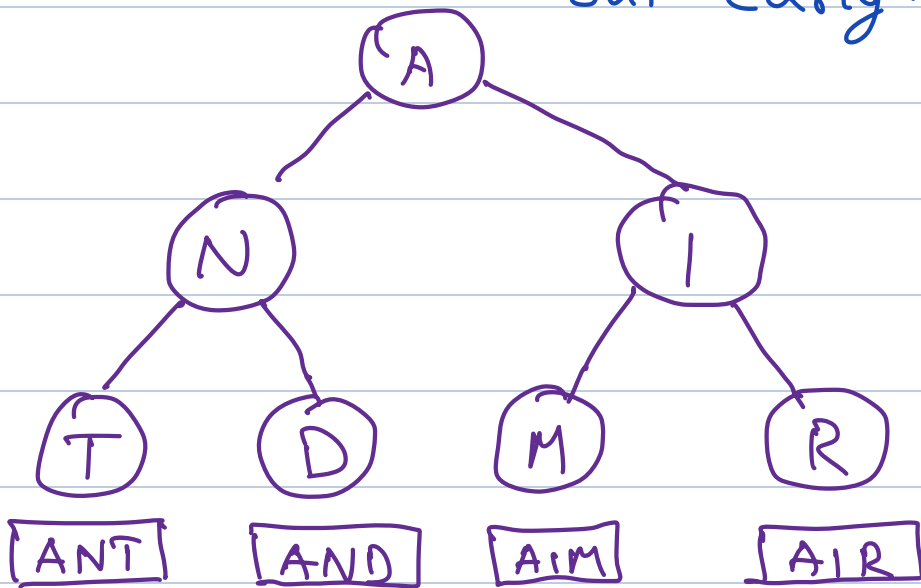
$$T.C = O(\# \text{ func calls} * TC \text{ of each})$$

$$= O(\log N * 1)$$

$$= O(\log N)$$

$$S.C = O(\log N)$$

Backtracking : Traverse all paths but break out early.



AIR

Question: Valid Parenthesis

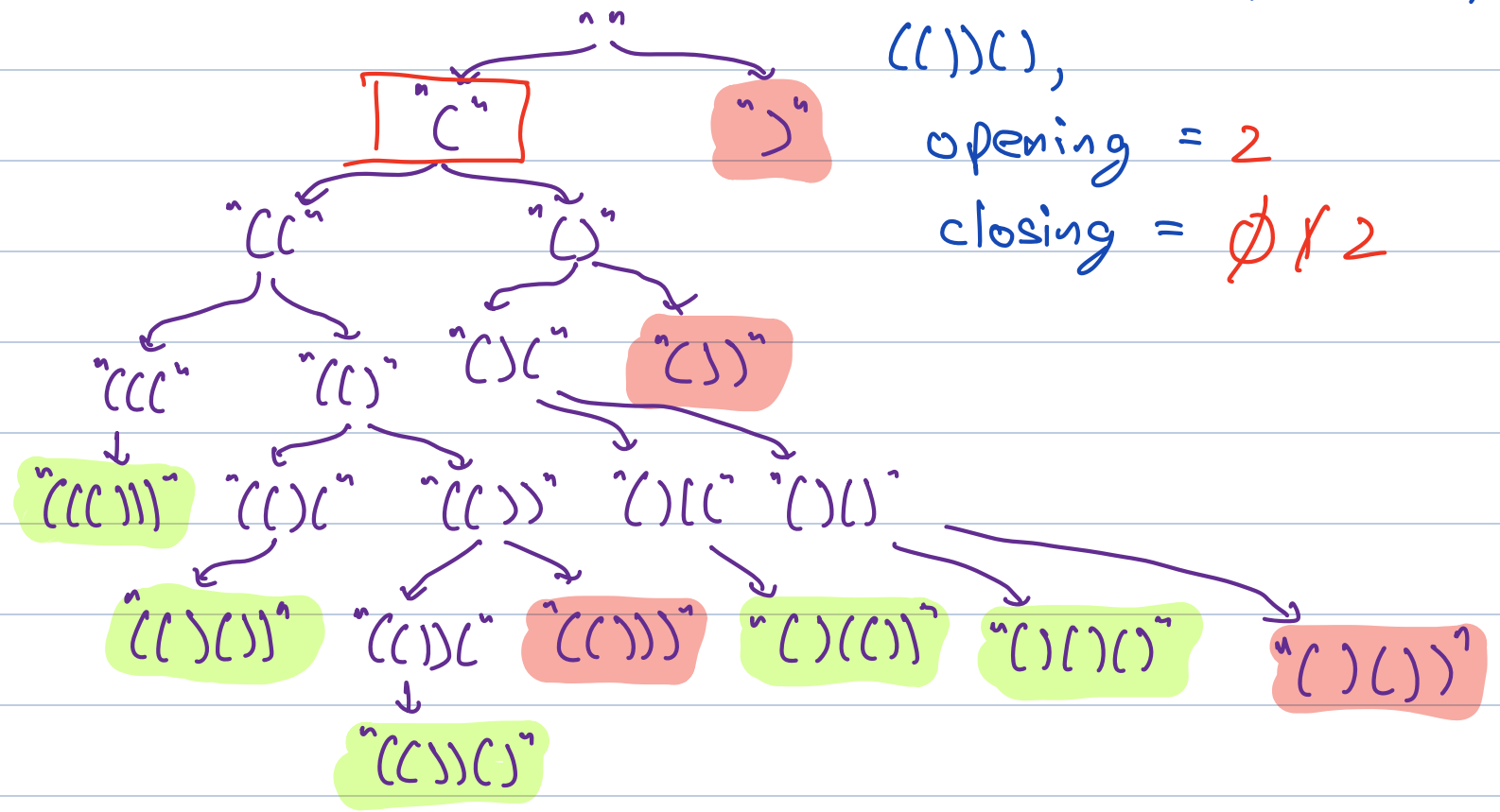
Given an integer N , write a function to generate all combinations of valid parathesis of length $2N$.

ex: $N = 3$

$$\text{ans} = [(c(c))), (c)(c), (c))(c),$$

opening = 2

closing = $\emptyset \neq 2$


$$\text{O/p} = [((c))', '(c)(c)', '(c)(c)', '(c)(c)', '(c)(c)(c)']$$

Observations:

- opening brackets count $< N$
 - ↳ Add "(" (opening bracket)
- closing brackets $<$ opening brackets

↳ Add ")" (closing bracket)

```
func validParen (String str, int N, int opening,  
                int closing, String[] ans) {
```

```
    // Base case
```

```
    if (opening + closing == 2 * N) {  
        ans.add(str);  
        return;  
    }
```

```
    // Recurrence relation
```

```
    // add opening bracket
```

```
    if (opening < N) {  
        validParen (str + "(", N, opening + 1,  
                    closing, ans);  
    }
```

```
    // add closing bracket.
```

```
    if (closing < opening) {  
        validParen (str + ")", N, opening,  
                    closing + 1, ans);  
    }
```

```
}
```

T.C = $O(2^N)$
S.C = $O(N)$

Subset: part of the main array but non-contiguous (order is maintained)

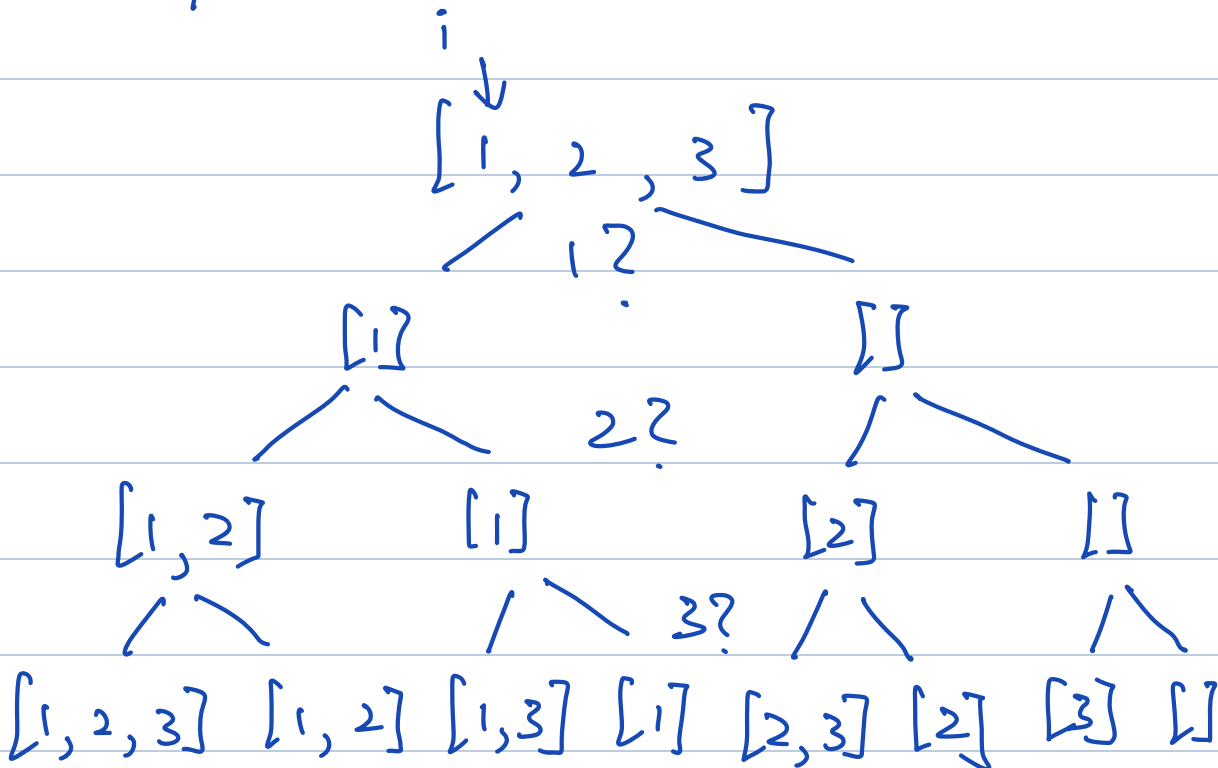
Question: All subsets

Given an array of distinct integers, print all the subsets of the array.

ex: $arr = [1, 2, 3]$

subsets = $[[], [1], [2], [3], [1, 2], [1, 3], [2, 3], [1, 2, 3]]$

of subsets = 2^N



Recurrence relation

- 1) Take current element $\rightarrow$ Add to the list
- 2) Don't take current element $\rightarrow$ Do nothing.

```
func recurse (int[] cur, int[] arr, int i, int[][] ans)
// Base case
if (i == len(arr)) {
    ans.add(curr);
    return;
}
// Take
recurse (cur + arr[i], arr, i+1, ans);
// No take
recurse (cur, arr, i+1, ans);
}

main() {
    arr = {1, 2, 3};
    ans = {};
    recurse ({}, arr, 0, ans);
    return ans;
}
```

$$T.C = O(2^N)$$

$$S.C = O(N)$$

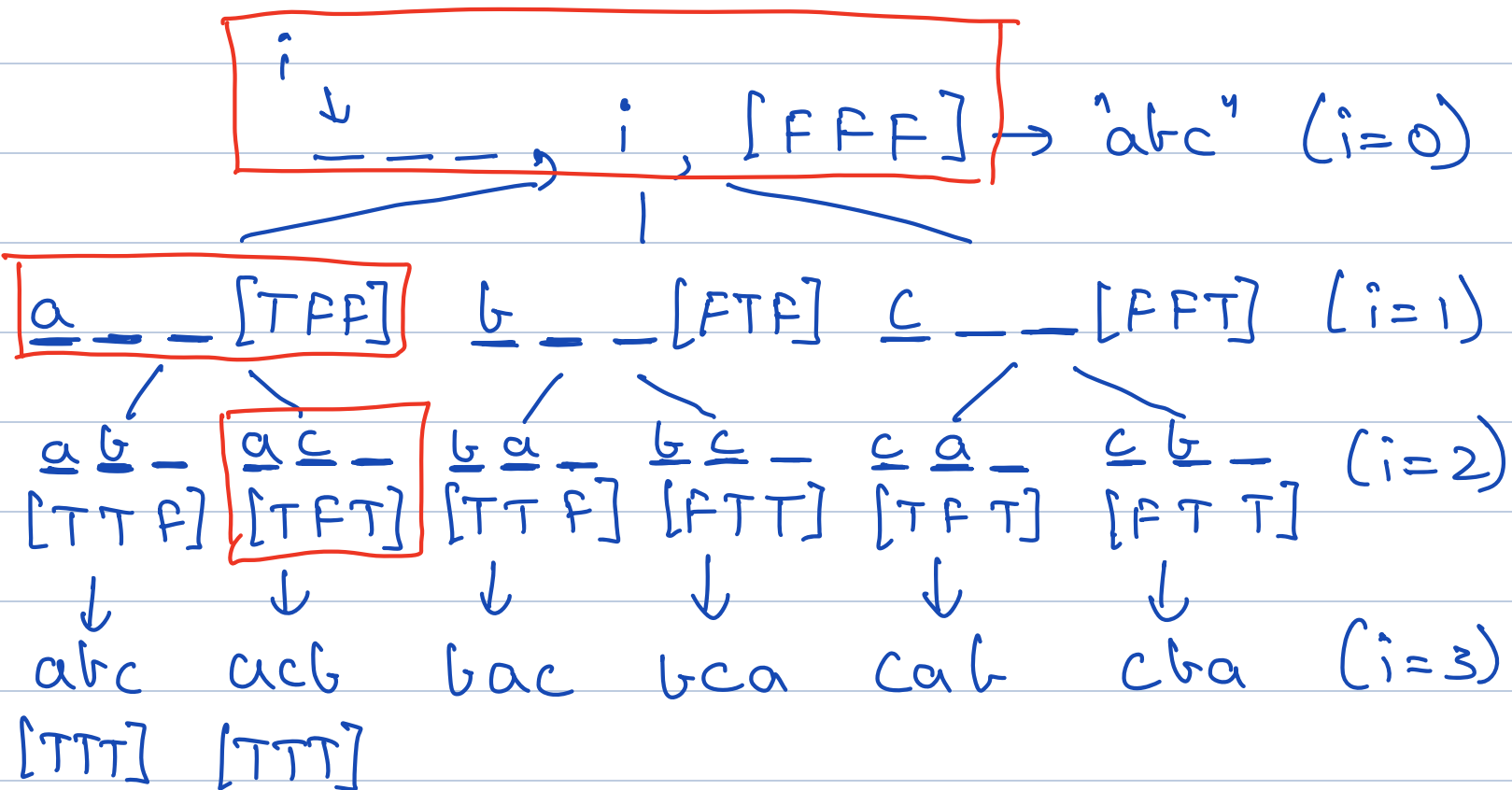
Permutations: Different ways of arranging the elements (order doesn't matter)
Size is fixed.

Question: All permutations

Given a string with distinct elements, print all permutations of it without modifying it.

ex: $s = "abc"$

All permutations $= ["abc", "acb", "bac", "bca", "cab", "cba"] = N!$



```
func allPerm(char[] curr, String str, int i,  
            bool[] visited, String[] ans){
```

```
// Base case
```

```
if (i == str.length){
```

```
    ans.add ( toString(curr));
```

```
    return;
```

```
}
```

```
// Iterate through visited array and  
    consider only un-visited elements.
```

```
for (j = 0, j < len(visited); j++){
```

```
    if (visited[j] == false){
```

```
        // Add jth element to curr
```

```
        curr.add(str[j]);
```

```
        // Mark jth element as visited
```

```
        visited[j] = true;
```

```
        // continue recursing
```

```
        allPerm(curr, str, i+1, visited, ans);
```

```
        // Mark as unvisited
```

```
        visited[j] = false;
```

```
        // Remove last element from curr
```

```
        curr.pop();
```

| }

}

}

$$TC = O(N!)$$

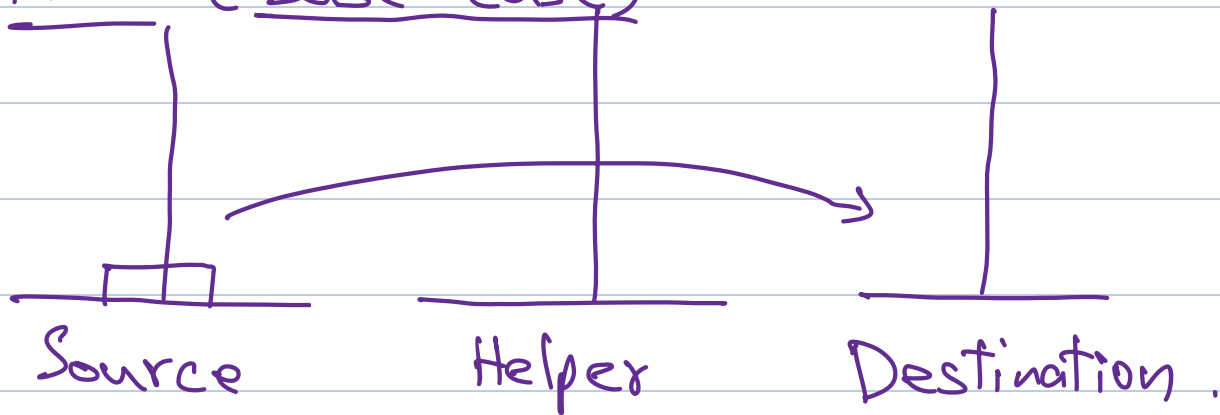
$$SC = O(N)$$

Question: Towers of Hanoi

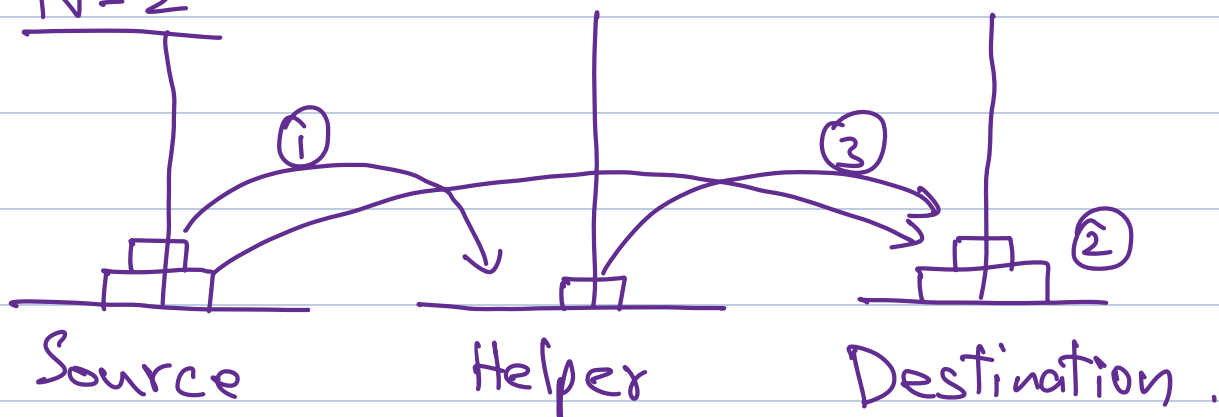
Transfer discs from source to destination in the same order. Use helper tower

Note: Larger disk cannot be placed on top of smaller tower.

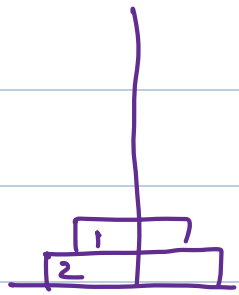
$N=1$ (Base Case)



$N=2$



Breakdown



Source



Helper



Destination.

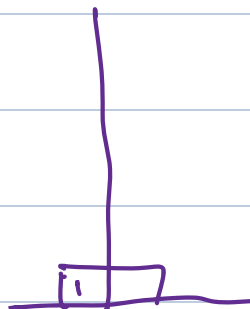
Step 1: Move Disc 1 from source to helper
Use Base case to move

Step 2: Move Disc 2 from source to destⁿ
Use base case to move

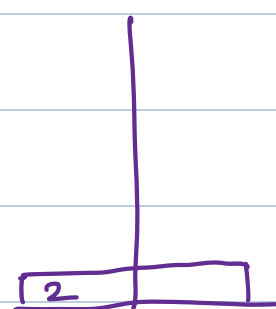
Step 3: Move Disc 1 from helper to destⁿ
Use base case to move.



Source



Helper



Destination

$N=3$



Step 1: Move top 2 discs ($N=2$) from source to helper.

Recurse with $N=2$

Step 2: Base case (3 from source to destination)

Step 3: Move $N=2$ discs from helper to dest.

Recurse with $N=2$.

```
func ToH (int N, int i, char s, char d, char h){  
    // Base case  
    if (N == 1) {  
        print ("Move Disc " + i + "from " + s +  
            "to " + d );  
        return;  
    }  
}
```

// Move $N-1$ discs from source to helper.

ToH($N-1, i, s, h, d$);

// Move N th disc from source to destⁿ.

ToH($1, N, s, d, h$);

// Move $N-1$ discs from helper to destⁿ

ToH($N-1, i, h, d, s$);

}

Doubts

Total Subarray OR Sum

011 001 101 010

[3, 1, 5, 2]

```
func solve (int arr) {  
    hm = precompute(arr);  
    total_subs = N*(N+1)/2  
    visited[32] = [False ... ]  
    ans = 0  
    for i → 0 to N:  
        for j → 0 to 31:  
            if (checkSetBit(arr[i], j) and visited[j] == F) {  
                visited[j] = True;  
                not_part_of = hm[j];  
                contri =  $2^j * (total\_subs - not\_part\_of)$ ;  
            }  
            ans += contri;  
    return ans;  
}
```

```
func precompute (int[] arr) {
```

```
    hm = {}
```

```
    for j → 0 to 31:
```

```
        indexes = []
```

```
        for i → 0 to N-1:
```

```
            if (!checkSetBit(arr[i], j)):
```

```
                indexes.append(i)
```

```
        total = 0, cnt = 1
```

```
        for i → 1 to indexes.length-1:
```

```
            if (indexes[i-1] + 1 == indexes[i])
```

```
                cnt += 1
```

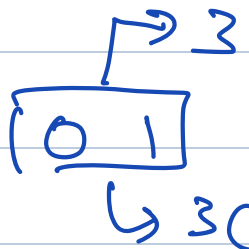
```
            else:
```

```
                total += cnt * (cnt + 1) / 2
```

```
        hm[j] = total.
```

```
    return hm;
```

```
}
```



$$\text{target} = 7 = 111$$

$$\begin{array}{ccc} \downarrow & \downarrow & \downarrow \\ 4 & 2 & 1 \end{array}$$

$$2^j = \textcircled{4} \rightarrow \text{Score} = 0$$

↳ Help from Sam = 1

↳ Solve yourself j times = $1 * 2^j$